Bharatiya Vidya Bhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (W), Mumbai – 400 058.
Department of Computer Engineering

Lab Assignment

Aim: Building and Securing a Two-Tier PKI with X.509

1. Objectives

By the end of this lab you will be able to:

1. Build a two-tier Certificate Authority (Root CA and Intermediate CA) using OpenSSL and configure X.509 v3 extensions correctly.
2. Issue server and client certificates from CSRs and verify the chain of trust.
3. Configure a web server for mutual TLS (mTLS) and test it with valid and invalid client certificates.
4. Revoke a certificate and enforce revocation using both a CRL and an OCSP responder.
5. Capture and analyse TLS 1.2 and TLS 1.3 handshakes in Wireshark and explain what is visible on the wire.

2. Environment and Prerequisites

Item	Requirement
OS	Ubuntu 22.04 / 24.04 recommended; 18.04 works with the notes in Section 2.1 (VM or native)
Tools	`openssl` 1.1.1 or later, `nginx`, `curl`, `wireshark` (or `tshark`)
Background	Public-key cryptography, digital signatures, TLS basics

Install the tools:

```
sudo apt update
sudo apt install -y openssl nginx curl wireshark
```

Add a hostname for the lab server:

```
echo '127.0.0.1 server.lab.local' | sudo tee -a /etc/hosts
```

Safety note: Private keys in this lab are stored unencrypted for convenience. In real deployments, a root CA key is encrypted and kept offline. Never reuse these keys or CAs for anything real, and never install the lab Root CA into your browser's permanent trust store.

2.1 Ubuntu Version Compatibility

Ubuntu 22.04 and 24.04 are recommended. Ubuntu 18.04 LTS also works for Parts A to E, but it is out of standard support (ended May 2023) and its Wireshark is too old to decrypt TLS 1.3, so Part F needs the workarounds below.

Component	Ubuntu 18.04	Ubuntu 22.04 / 24.04	What to do on 18.04
OpenSSL	1.1.1	3.0	Nothing; every command in this manual works on both
nginx	1.14	1.18 / 1.24	Nothing; this manual does not use the `-e` flag (added in nginx 1.19.5)
curl	7.58 (no reliable TLS 1.3)	7.81 or later	Use `openssl s_client -tls1_3` for TLS 1.3 tests (Part F.2)
Wireshark	2.6 (filters start with `ssl.`)	3.6 / 4.2 (filters start with `tls.`)	Install 3.0 or later from the official PPA to do F.3 (below), or skip F.3

To install a newer Wireshark on 18.04:

```
sudo add-apt-repository ppa:wireshark-dev/stable
sudo apt update
sudo apt install -y wireshark
```

3. Lab Overview

Part	Topic
A	Create the Root CA
B	Create the Intermediate CA
C	Issue server and client certificates
D	Mutual TLS with nginx
E	Revocation: CRL and OCSP
F	Wireshark TLS handshake analysis
G	Break it: failure scenarios

Set a working-directory variable and create the folder layout. **Run `export PKI=$HOME/pki-lab` in every new terminal.**

```
export PKI=$HOME/pki-lab
mkdir -p $PKI/{root,intermediate,server,client,nginx,captures}
cd $PKI
mkdir -p root/{certs,crl,newcerts,private}
mkdir -p intermediate/{certs,crl,csr,newcerts,private}
chmod 700 root/private intermediate/private
touch root/index.txt intermediate/index.txt
echo 1000 > root/serial
echo 1000 > intermediate/serial
echo 1000 > root/crlnumber
echo 1000 > intermediate/crlnumber
```

Part A: Create the Root CA

A.1 Configuration

Create `$PKI/root/openssl.cnf` with the following content:

```
[ ca ]
default_ca = CA_default

[ CA_default ]
dir               = $ENV::PKI/root
certs             = $dir/certs
crl_dir           = $dir/crl
new_certs_dir     = $dir/newcerts
database          = $dir/index.txt
serial            = $dir/serial
crlnumber         = $dir/crlnumber
private_key       = $dir/private/root.key.pem
certificate       = $dir/certs/root.cert.pem
crl               = $dir/crl/root.crl.pem
default_md        = sha256
default_days      = 365
default_crl_days  = 30
policy            = policy_loose
unique_subject    = no
copy_extensions   = none

[ policy_loose ]
countryName            = optional
organizationName       = optional
organizationalUnitName = optional
commonName             = supplied

[ req ]
default_md        = sha256
distinguished_name = req_dn
prompt            = no

[ req_dn ]
CN = placeholder

[ v3_root_ca ]
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid:always,issuer
basicConstraints       = critical, CA:true
keyUsage               = critical, keyCertSign, cRLSign

[ v3_intermediate_ca ]
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid:always,issuer
basicConstraints       = critical, CA:true, pathlen:0
keyUsage               = critical, keyCertSign, cRLSign

[ crl_ext ]
authorityKeyIdentifier = keyid:always
```

A.2 Generate the key and self-signed certificate

```
cd $PKI
openssl genrsa -out root/private/root.key.pem 4096
chmod 400 root/private/root.key.pem

openssl req -config root/openssl.cnf -new -x509 -days 3650 -sha256 \
  -extensions v3_root_ca \
  -key root/private/root.key.pem \
  -subj '/O=CSE Lab/CN=CSE Lab Root CA' \
  -out root/certs/root.cert.pem
```

A.3 Inspect

```
openssl x509 -in root/certs/root.cert.pem -noout -text
```

Task A: Record the serial number, validity period, signature algorithm, key size, and every extension. Explain why `basicConstraints` is marked **critical** and why a Root CA is self-signed (issuer equals subject).

Part B: Create the Intermediate CA

B.1 Configuration

Create `$PKI/intermediate/openssl.cnf`:

```
[ ca ]
default_ca = CA_default

[ CA_default ]
dir               = $ENV::PKI/intermediate
certs             = $dir/certs
crl_dir           = $dir/crl
new_certs_dir     = $dir/newcerts
database          = $dir/index.txt
serial            = $dir/serial
crlnumber         = $dir/crlnumber
private_key       = $dir/private/intermediate.key.pem
certificate       = $dir/certs/intermediate.cert.pem
crl               = $dir/crl/intermediate.crl.pem
default_md        = sha256
default_days      = 365
default_crl_days  = 30
policy            = policy_loose
unique_subject    = no
copy_extensions   = none

[ policy_loose ]
countryName           = optional
organizationName      = optional
organizationalUnitName = optional
commonName            = supplied

[ req ]
default_md        = sha256
distinguished_name = req_dn
prompt            = no
```

```
[ req_dn ]
CN = placeholder

[ server_cert ]
basicConstraints       = CA:FALSE
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid,issuer:always
keyUsage               = critical, digitalSignature, keyEncipherment
extendedKeyUsage       = serverAuth
subjectAltName         = @alt_names
authorityInfoAccess    = OCSP;URI:http://127.0.0.1:2560

[ client_cert ]
basicConstraints       = CA:FALSE
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid,issuer:always
keyUsage               = critical, digitalSignature
extendedKeyUsage       = clientAuth
authorityInfoAccess    = OCSP;URI:http://127.0.0.1:2560

[ alt_names ]
DNS.1 = server.lab.local
DNS.2 = localhost
IP.1  = 127.0.0.1

[ crl_ext ]
authorityKeyIdentifier = keyid:always
```

B.2 Generate the key and CSR, then sign with the Root CA

```
cd $PKI
openssl genrsa -out intermediate/private/intermediate.key.pem 4096
chmod 400 intermediate/private/intermediate.key.pem

openssl req -config intermediate/openssl.cnf -new -sha256 \
  -key intermediate/private/intermediate.key.pem \
  -subj '/O=CSE Lab/CN=CSE Lab Intermediate CA' \
  -out intermediate/csr/intermediate.csr.pem

openssl ca -batch -config root/openssl.cnf -extensions v3_intermediate_ca \
  -days 1825 -notext -md sha256 \
  -in intermediate/csr/intermediate.csr.pem \
  -out intermediate/certs/intermediate.cert.pem
```

B.3 Build the chain file and verify

```
cat intermediate/certs/intermediate.cert.pem root/certs/root.cert.pem \
  > intermediate/certs/ca-chain.cert.pem

openssl verify -CAfile root/certs/root.cert.pem intermediate/certs/intermediate.cert.pem
```

Task B: Compare the Root and Intermediate certificates. What does `pathlen:0` do? What would happen if you issued a certificate *from* a leaf certificate?

Part C: Issue Server and Client Certificates

C.1 Server certificate

```
cd $PKI
openssl genrsa -out server/server.key.pem 2048
openssl req -new -key server/server.key.pem \
  -subj '/O=CSE Lab/CN=server.lab.local' -out server/server.csr.pem

openssl ca -batch -config intermediate/openssl.cnf -extensions server_cert \
  -days 365 -notext -md sha256 \
  -in server/server.csr.pem -out server/server.cert.pem

cat server/server.cert.pem intermediate/certs/intermediate.cert.pem > server/server.fullchain.pem
```

C.2 Two client certificates (Alice and Bob)

```
for u in alice bob; do
  openssl genrsa -out client/$u.key.pem 2048
  openssl req -new -key client/$u.key.pem \
    -subj "/O=CSE Lab/CN=$u" -out client/$u.csr.pem
  openssl ca -batch -config intermediate/openssl.cnf -extensions client_cert \
    -days 365 -notext -md sha256 \
    -in client/$u.csr.pem -out client/$u.cert.pem
done
```

C.3 Verify

```
openssl verify -CAfile intermediate/certs/ca-chain.cert.pem server/server.cert.pem
openssl verify -CAfile intermediate/certs/ca-chain.cert.pem client/alice.cert.pem
openssl verify -CAfile intermediate/certs/ca-chain.cert.pem -purpose sslclient client/alice.cert.pem
```

Task C:

1. Inspect `intermediate/index.txt`. What do the columns mean (status, expiry, serial, subject)?
2. Show that Alice's certificate fails verification with `-purpose sslserver`. Which extension causes this?
3. Why does the server need a Subject Alternative Name even though the CN already contains the hostname?

Part D: Mutual TLS with nginx

D.1 nginx configuration

Create `$PKI/nginx/nginx.conf`. **Replace `/home/student` with your actual home path** (nginx does not expand variables in paths).

```
pid /home/student/pki-lab/nginx/nginx.pid;
error_log /home/student/pki-lab/nginx/error.log info;
events {}
http {
  access_log /home/student/pki-lab/nginx/access.log;
  server {
    listen 8443 ssl;
    server_name server.lab.local;

    ssl_certificate         /home/student/pki-lab/server/server.fullchain.pem;
```

```
    ssl_certificate_key     /home/student/pki-lab/server/server.key.pem;
    ssl_protocols           TLSv1.2 TLSv1.3;

    ssl_client_certificate  /home/student/pki-lab/intermediate/certs/ca-chain.cert.pem;
    ssl_verify_client       on;
    ssl_verify_depth        2;

    location / {
      return 200 'Hello $ssl_client_s_dn | verify=$ssl_client_verify | proto=$ssl_protocol';
    }
  }
}
```

Start nginx with your own config (no root privileges needed on port 8443):

```
nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf
```

Note: nginx may print a harmless warning about not being able to open `/var/log/nginx/error.log` at start-up. It then logs to the `error_log` path in your config file. To stop it, run `nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s stop`.

D.2 Test

```
# Valid client
curl --cacert intermediate/certs/ca-chain.cert.pem \
  --cert client/alice.cert.pem --key client/alice.key.pem \
  https://server.lab.local:8443/

# No client certificate
curl --cacert intermediate/certs/ca-chain.cert.pem https://server.lab.local:8443/
```

See the handshake in detail with:

```
openssl s_client -connect server.lab.local:8443 -CAfile intermediate/certs/ca-chain.cert.pem \
  -cert client/alice.cert.pem -key client/alice.key.pem -showcerts
```

Task D:

1. Paste the output for both tests. What HTTP status does the no-certificate request receive, and why?
2. Which header or log line identifies the connecting client?
3. Why does nginx need `ssl_client_certificate` and `ssl_verify_depth 2`?

Part E: Revocation (CRL and OCSP)

E.1 Revoke Bob's certificate and publish a CRL

```
cd $PKI
openssl ca -config intermediate/openssl.cnf -revoke client/bob.cert.pem -crl_reason keyCompromise

openssl ca -config intermediate/openssl.cnf -gencrl -out intermediate/crl/intermediate.crl.pem
openssl ca -config root/openssl.cnf -gencrl -out root/crl/root.crl.pem

cat intermediate/crl/intermediate.crl.pem root/crl/root.crl.pem > intermediate/crl/combined.crl.pem
openssl crl -in intermediate/crl/intermediate.crl.pem -noout -text
```

Check revocation with OpenSSL:

```
openssl verify -crl_check -CAfile intermediate/certs/ca-chain.cert.pem \
  -CRLfile intermediate/crl/combined.crl.pem client/bob.cert.pem
```

E.2 Enforce the CRL in nginx

Add this line inside the `server` block and reload:

```
ssl_crl /home/student/pki-lab/intermediate/crl/combined.crl.pem;
```

```
nginx -p $PKI/nginx -c $PKI/nginx/nginx.conf -s reload
```

Test again with Alice (should succeed) and Bob (should fail).

> ***Why a combined CRL?*** *nginx checks CRLs for every certificate in the chain. Without a CRL from the Root CA, you will see the error unable to get certificate CRL. Try removing `root.crl.pem` from the file and observe the failure.*

E.3 Run an OCSP responder

In a **separate terminal**:

```
export PKI=$HOME/pki-lab; cd $PKI
openssl ocsp -index intermediate/index.txt -port 2560 \
  -rsigner intermediate/certs/intermediate.cert.pem \
  -rkey intermediate/private/intermediate.key.pem \
  -CA intermediate/certs/ca-chain.cert.pem -text
```

Query it:

```
openssl ocsp -issuer intermediate/certs/intermediate.cert.pem \
  -cert client/alice.cert.pem -url http://127.0.0.1:2560 \
  -CAfile intermediate/certs/ca-chain.cert.pem -resp_text

openssl ocsp -issuer intermediate/certs/intermediate.cert.pem \
  -cert client/bob.cert.pem -url http://127.0.0.1:2560 \
  -CAfile intermediate/certs/ca-chain.cert.pem -resp_text
```

Note: Using the CA key itself as the OCSP signer is a lab simplification. Production systems use a dedicated certificate with the `OCSPSigning` extended key usage.

Task E:

1. Show the status returned for Alice and Bob, including the revocation time and reason.
2. Compare CRL and OCSP in a table: freshness, size, privacy, latency, failure behaviour (soft-fail vs hard-fail).
3. What is OCSP stapling, and which problem with plain OCSP does it solve?
4. Where in the server certificate is the OCSP URL stored? (Hint: look for Authority Information Access.)

Part F: Wireshark TLS Handshake Analysis

> ***Ubuntu 18.04 users:*** *Wireshark 2.6 labels the protocol SSL, so use the filters `ssl.handshake` and `ssl.handshake.type == 11`, and find the key log setting under Protocols > SSL. Its TLS 1.3 decoding is limited, so F.3 needs Wireshark 3.0 or later (see Section 2.1).*

F.1 Capture a TLS 1.2 handshake

Start Wireshark on the **loopback (lo)** interface with capture filter `tcp port 8443`. Un-revoke is not needed; use Alice's certificate.

```
curl --tlsv1.2 --tls-max 1.2 --cacert intermediate/certs/ca-chain.cert.pem \
  --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
```

Stop the capture and save it as `captures/tls12.pcapng`. Use the display filter `tls.handshake`.

1. Find the **ClientHello** (note the SNI and cipher suites) and **ServerHello**.
2. Filter `tls.handshake.type == 11` to find the **Certificate** messages. Expand one and locate the subject, issuer, validity, and extensions.
3. Find the **CertificateRequest** (server asking for a client certificate) and the client's **Certificate** and **CertificateVerify** messages.

F.2 Capture a TLS 1.3 handshake

```
curl --tlsv1.3 --cacert intermediate/certs/ca-chain.cert.pem \
  --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
```

Ubuntu 18.04 alternative: curl 7.58 may not support TLS 1.3 with OpenSSL. Use OpenSSL's own client instead:

```
openssl s_client -connect server.lab.local:8443 -tls1_3 \
  -CAfile intermediate/certs/ca-chain.cert.pem \
  -cert client/alice.cert.pem -key client/alice.key.pem
```

Press Ctrl+C after the handshake completes.

Save as `captures/tls13.pcapng` and repeat the filters. In TLS 1.3, the Certificate message is **encrypted** and will not appear.

F.3 Decrypt TLS 1.3 using a key log

```
export SSLKEYLOGFILE=$PKI/captures/keys.log
curl --tlsv1.3 --cacert intermediate/certs/ca-chain.cert.pem \
  --cert client/alice.cert.pem --key client/alice.key.pem https://server.lab.local:8443/
```

In Wireshark: *Edit > Preferences > Protocols > TLS > (Pre)-Master-Secret log filename*, and select `keys.log`. Reload the capture; the Certificate messages should now be decoded.

If you are using `openssl s_client` instead of curl, add `-keylogfile $PKI/captures/keys.log` to the command rather than setting `SSLKEYLOGFILE`.

Task F:

1. Fill in a table comparing TLS 1.2 and TLS 1.3 for: number of round trips, visibility of server certificate, visibility of client certificate, key exchange messages.
2. Why is it a privacy improvement that the client certificate is encrypted in TLS 1.3? What does a passive observer learn in TLS 1.2 that they cannot learn in TLS 1.3?
3. Which handshake message proves the client possesses the private key matching its certificate?
4. Include screenshots with the relevant packets highlighted.

Part G: Break It (Failure Scenarios)

For each scenario, create the failing condition, run the test, capture the exact error (curl, OpenSSL, and nginx `error.log`), and explain the root cause in one or two sentences.

No.	Scenario	Hint
1	**Expired client certificate**	Sign with `openssl ca ... -startdate 20240101000000Z -enddate 20240102000000Z`
2	**Untrusted client**	Create a self-signed certificate for Mallory and try to connect
3	**Wrong purpose**	Use the *server* certificate as a client certificate
4	**Hostname mismatch**	Connect with `--resolve wrong.lab.local:8443:127.0.0.1` to `https://wrong.lab.local:8443/`
5	**Missing intermediate**	Use `server.cert.pem` alone (without the intermediate) as `ssl_certificate`; compare curl with and without `--cacert` set to the chain
6	**Revoked certificate**	Already done with Bob; document the error in nginx logs

4. Submission

Submit a single PDF report plus a zip file containing:

- **Report** with sections for each Part (A to G), command outputs, screenshots, and answers to all Tasks.
- `root/openssl.cnf` and `intermediate/openssl.cnf` (no private keys).
- All **certificates** (`*.cert.pem`), the CA chain file, and both CRLs.
- `nginx.conf` and relevant `error.log` excerpts.
- `tls12.pcapng`, `tls13.pcapng`, and `keys.log`.
- A short **reflection** (150 to 250 words): *What is the single most common reason real-world TLS deployments fail certificate validation, and how would you prevent it?*

Do not submit private keys.

5. Troubleshooting

Symptom	Likely cause and fix
`unable to open index.txt.attr` or similar	Re-run the `touch`/`echo` setup commands; confirm `$PKI` is exported
`The commonName field needed to be supplied`	Pass `-subj` with a `CN=` value
`openssl ca` says certificate already exists	We set `unique_subject = no`; check the config was read with `-config`
nginx: `bind() failed (98: Address in use)`	Another instance is running; stop it with `-s stop` using the same `-p`/`-c` options
nginx: `unable to get certificate CRL`	CRL file is missing the Root CA's CRL; use the combined file
curl: `SSL certificate problem: unable to get local issuer certificate`	Pass `--cacert` with the full chain file
Wireshark shows no packets	Capture on `lo`, not your Ethernet or Wi-Fi interface